

Community Detection

Prof. O-Joun Lee

Dept. of Artificial Intelligence,
The Catholic University of Korea
ojlee@catholic.ac.kr

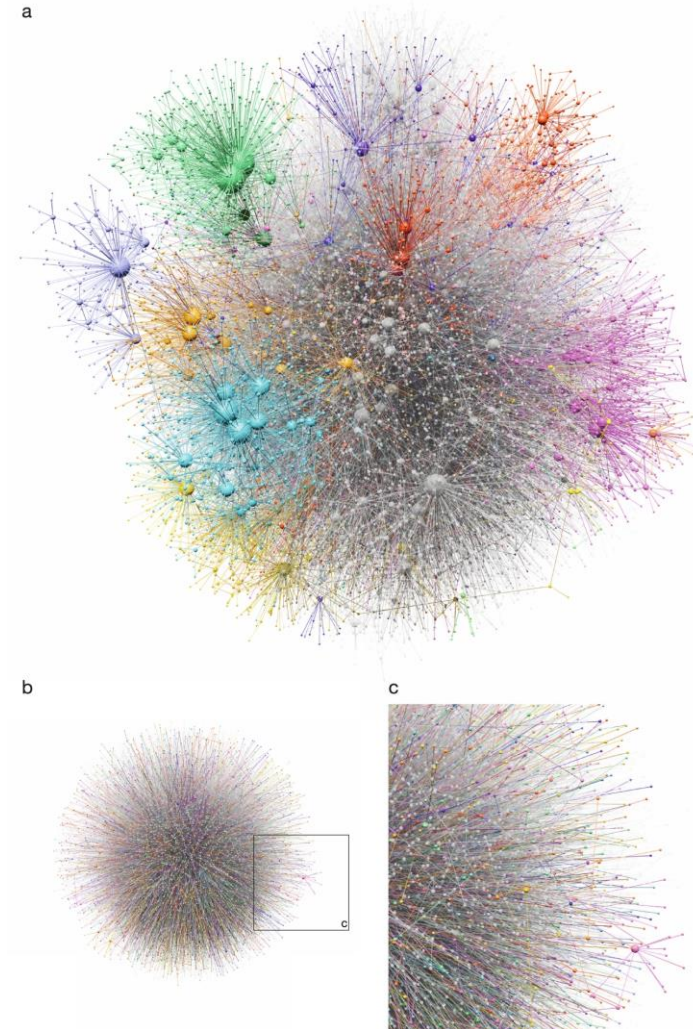
Contents



- Definition of communities and their properties
 - Definition and characteristic of communities
 - Example
- Clustering techniques
 - Maximum Clique, Clique Percolation Method, k-clique
 - Min-cut, Normalized-cut
- Modularity and its variants
- Tool for community detection
 - Hierarchy-Centric Decisive, Louvain, Leiden, and others
- Community evaluation

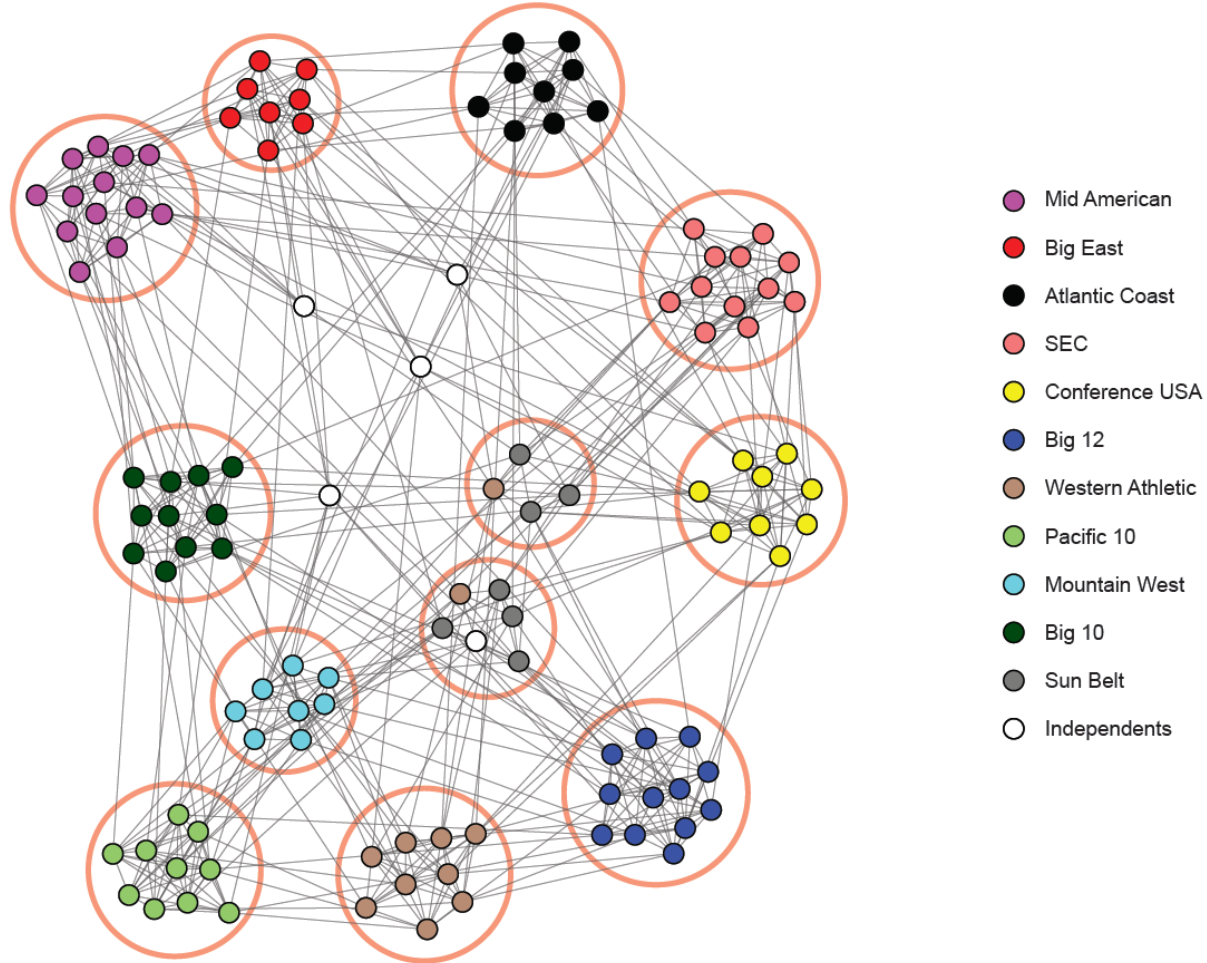
- Understand why and how community detection and validation work:
 - Explain the connection to modularity
- Distinguish methodologies used for overlapping and non-overlapping community detection
- Contrast methodology used in networks
- Evaluate the community detection

- Look at the internet network in the World:



- ➡ Too many information
- ➡ Do not know what to extract

➤ Look at the internet network in the World:



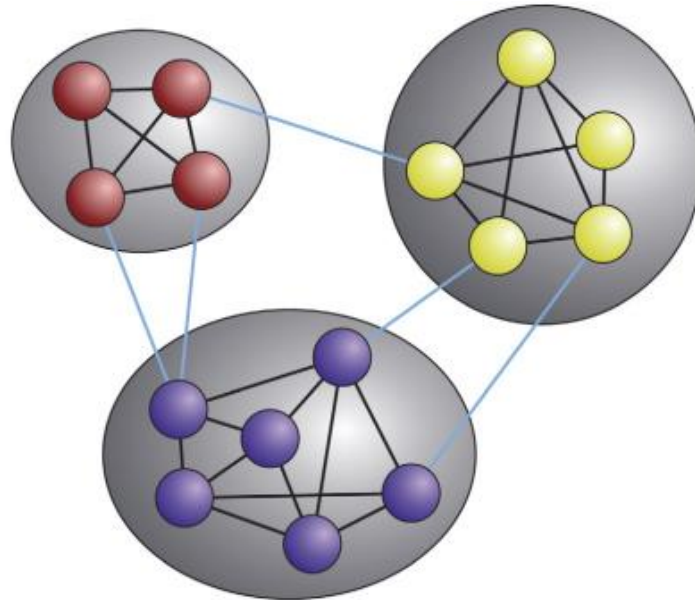
➡ Easy to recognition

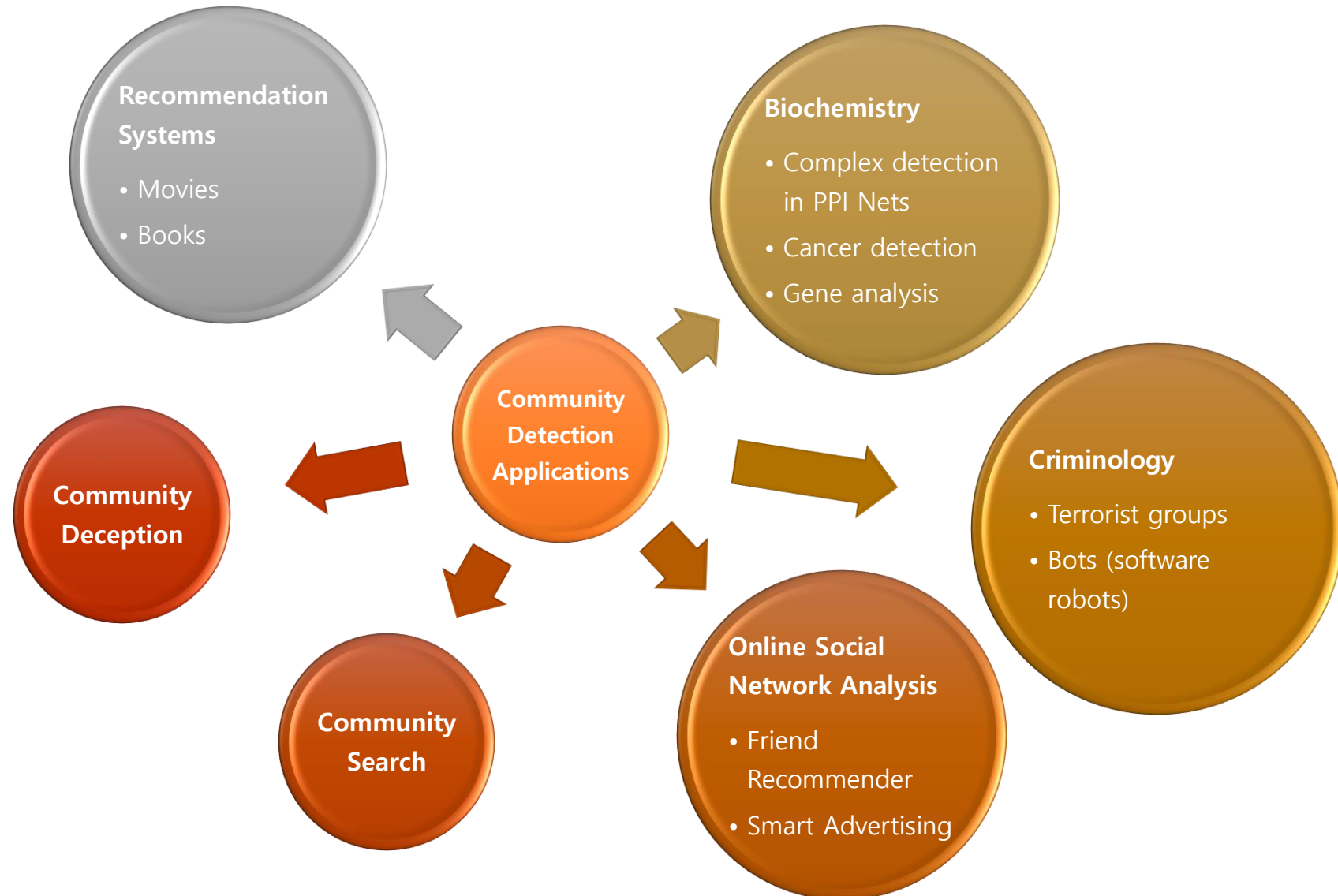
➤ What is difference?

➡ There are some similarity in each circle

➡ Those similar features make a circular community

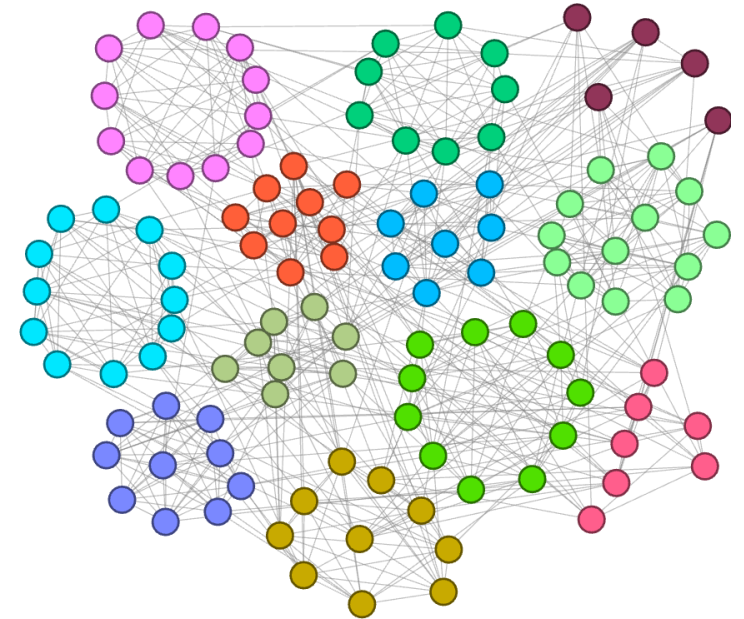
- Communities are features that appear in real networks
 - We generally try to identify them through the structural properties of the network: nodes tend to cluster based on common interests
- Based on its usefulness, community detection became one of the most prominent directions of research in network science.
- It is one of the common analysis tools in understanding networks





Source: Su, Xing, et al. "A comprehensive survey on community detection with deep learning." *IEEE transactions on neural networks and learning systems* 35.4 (2022): 4682-4702.

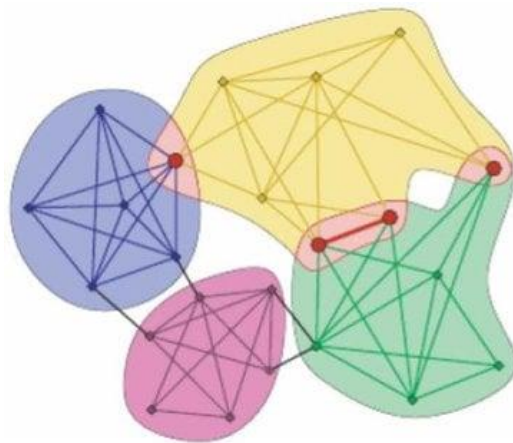
- Social science: A group of people with common characteristic or shared interests
- Graph perspective: a collection of nodes that have strong inter-connection within the community than what we would expect to occur at random
- Given a graph $G = (V, E)$, community detection consist in partitioning the set of vertices V into k subsets:
 - $P = \{C_1, C_2, \dots, C_k\}$ such that:
 - $\{C_1 \cup C_2 \cup \dots \cup C_k\} = V$ and $\{C_1 \cap C_2 \cap \dots \cap C_k\} = \emptyset$



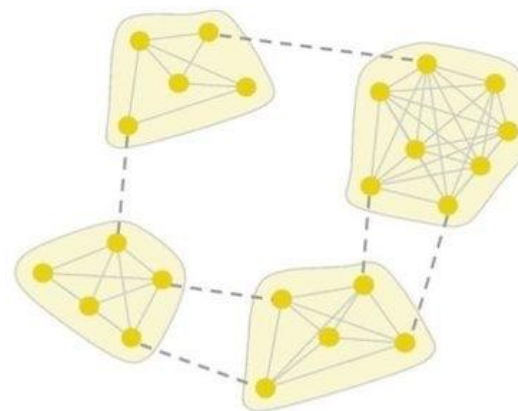
➤ Definition: Community

Given a set of communities $\mathcal{C} = \{C_1, C_2, \dots, C_K\}$, each community C_k is a partition of $\mathcal{G}$ which keeps regional structure and cluster properties. A node v_i clustered into community C_k should satisfy the condition that the internal node degree inside the community exceeds its external degree.

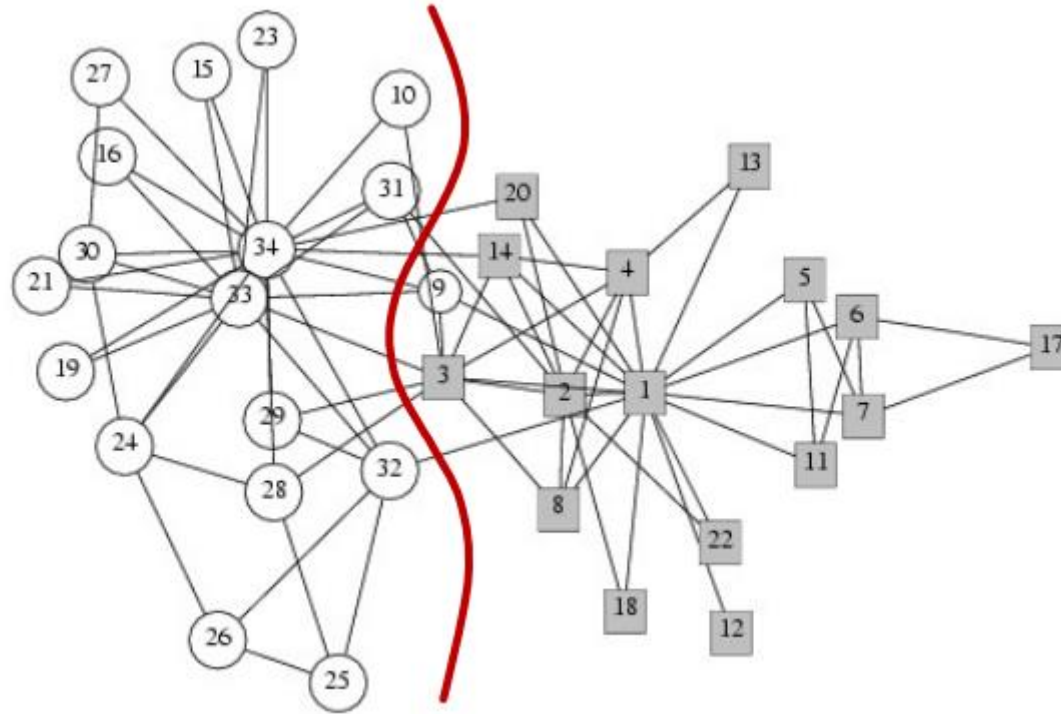
Suppose $C_k \cap C_{k'} = \emptyset, (\forall k, k'), \mathcal{C}$ denotes **disjoint** communities; otherwise **overlapping communities**.



Overlapping

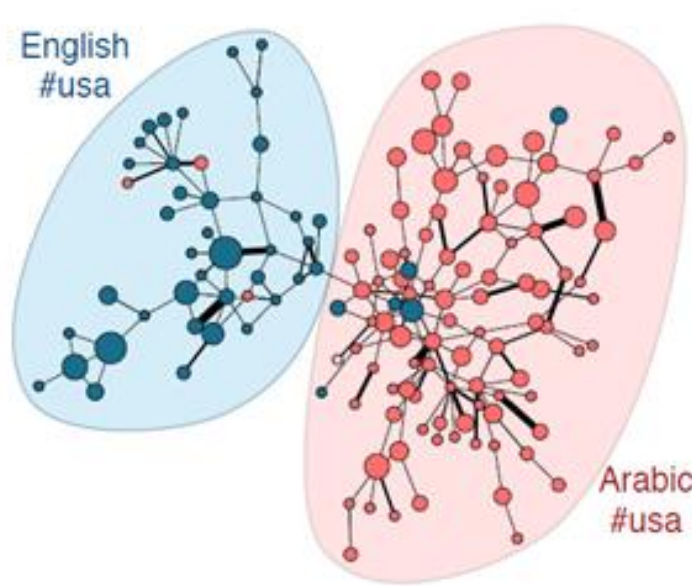


Disjoint

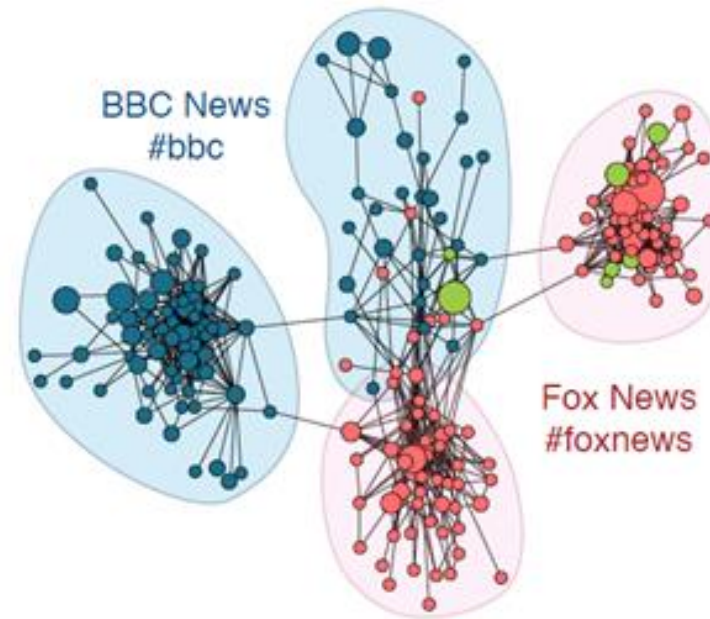


- Look at [Zachary's Karate club network](#):
 - Observed social ties & rivalries in a university karate club
 - During the study, conflicts led the group to split
 - Split could be explained by a minimum cut in the network

- **Homophily**: nodes with similar features tend to be connected within the same clusters.
 - For example, based on language (or based on degree for degree homophily)



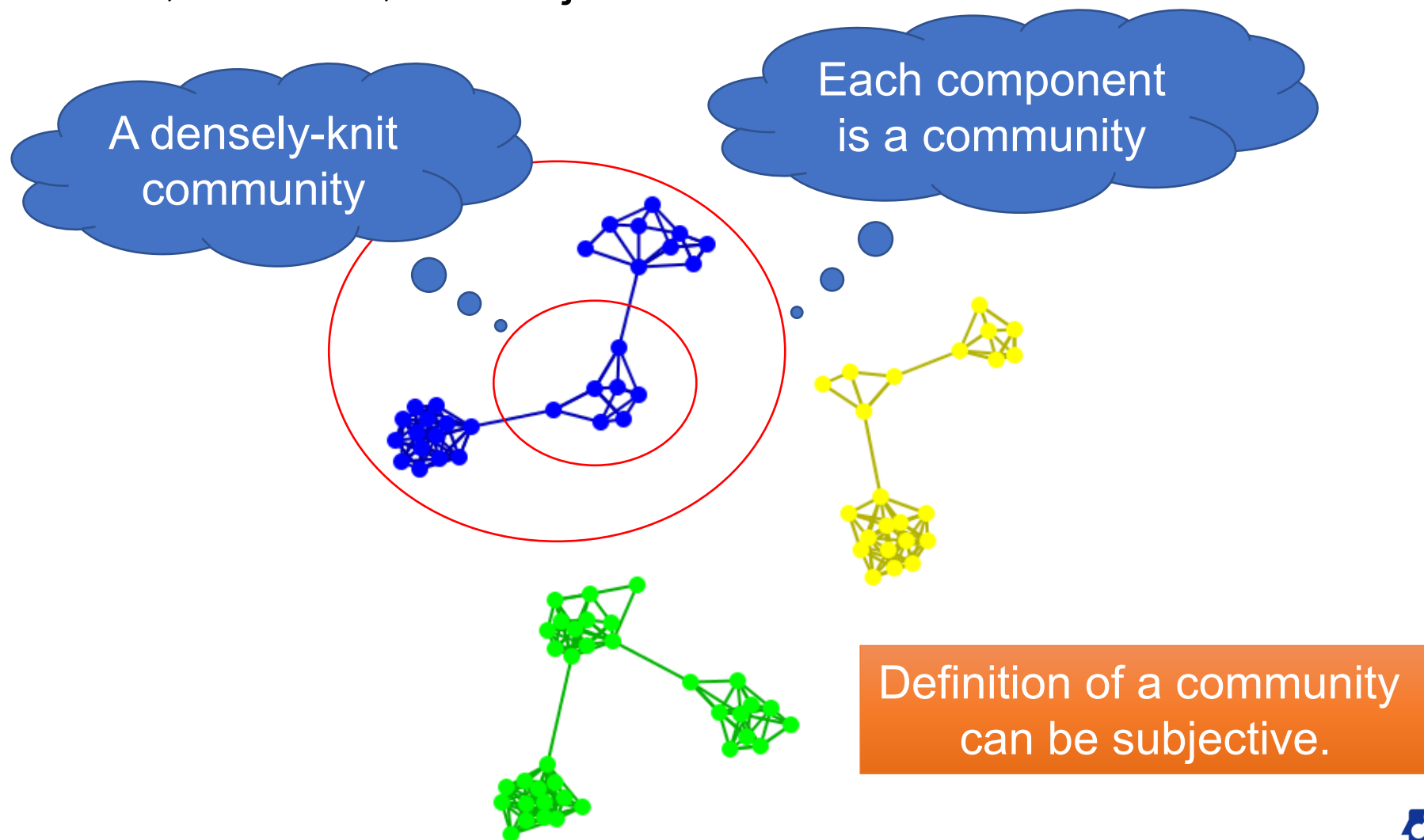
Retweet network



Follower network

- Two types of groups in social media:
 - Explicit Groups: formed by user subscriptions
 - Implicit Groups: implicitly formed by social interactions
- Some social media sites allow people to join groups, is it necessary to extract groups based on network topology?
 - Not all sites provide community platform
 - Not all people want to make effort to join groups
 - Groups can change dynamically
- Network interaction provides rich information about the relationship between users:
 - Can complement other kinds of information
 - Help network visualization and navigation
 - Provide basic information for other tasks

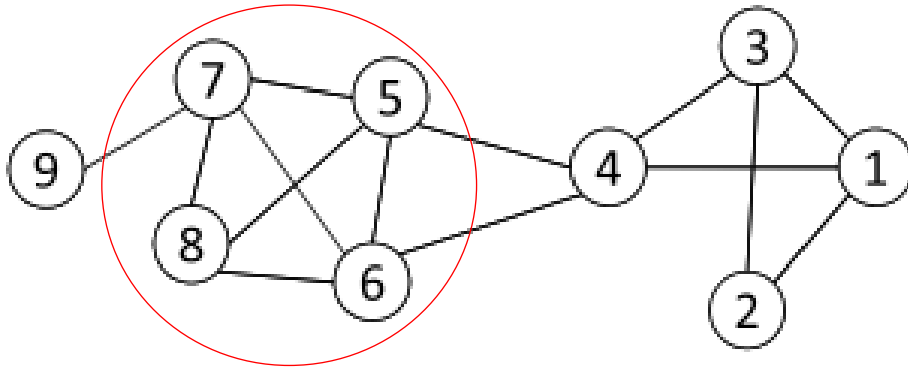
- What constitutes a community can be subjective and vary depending on perspectives, contexts, and objectives



- Criteria vary depending on the tasks
- Roughly, community detection methods can be divided into 4 categories (not exclusive):
 - 1. Node-Centric Community:
 - Each node in a group satisfies certain properties
 - 2. Group-Centric Community:
 - Consider the connections within a group as a whole. The group must satisfy certain properties without zooming into node-level
 - 3. Network-Centric Community:
 - Partition the whole network into several disjoint sets
 - 4. Hierarchy-Centric Community:
 - Construct a hierarchical structure of communities

- Nodes satisfy different properties
 - Complete Mutuality:
 - Cliques
 - Reachability of members:
 - k-clique, k-club
 - Nodal degrees:
 - k-plex, k-core
 - Relative frequency of Within-Outside Ties:
 - LS sets, Lambda sets
- Commonly used in traditional social network analysis

- Clique: a maximum complete subgraph in which all nodes are adjacent to each other

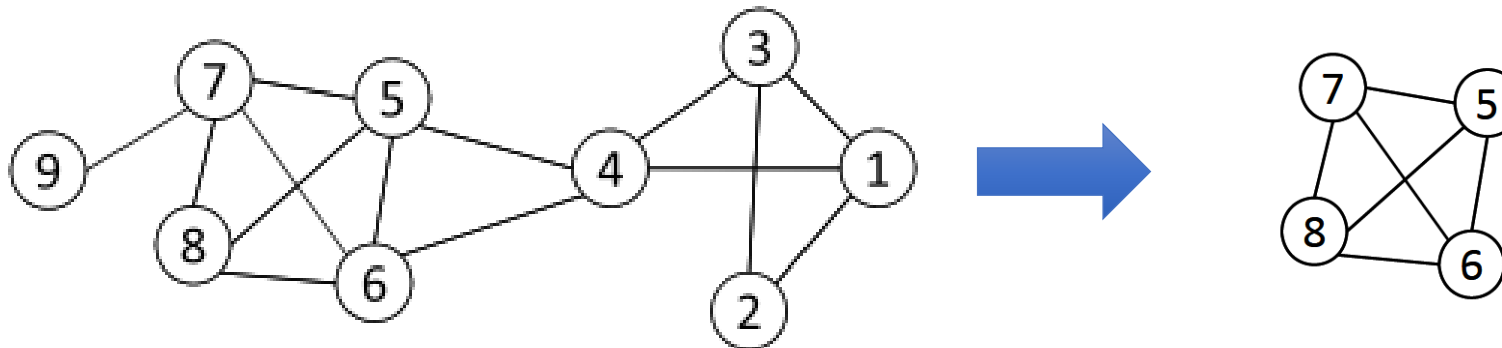


Nodes 5, 6, 7 and 8 form a clique

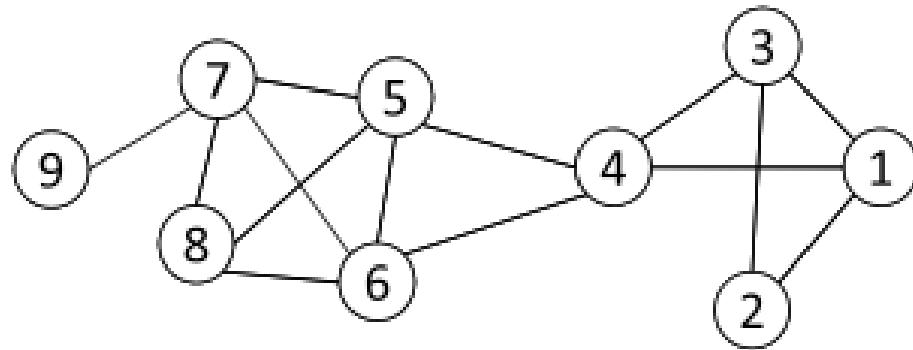
- NP-hard to find the maximum clique in a network
- Straightforward implementation to find cliques is very expensive in time complexity

- In a clique of size k , each node maintains degree $\geq (k - 1)$
- Nodes with degree $< (k - 1)$ will not be included in the maximum clique
- Recursively apply the following pruning procedure:
 - Sample a sub-network from the given network, and find a clique in the sub-network, say, by a greedy approach
 - Suppose the clique above is size k
 - To find out a larger clique, all nodes with degree $\leq (k - 1)$ should be removed
- Repeat until the network is small enough
- Many nodes will be pruned as social media networks follow a power law distribution for node degrees

- Suppose we sample a sub-network with nodes $\{1 - 5\}$ and find a clique $\{1, 2, 3\}$ of size 3
- To find a clique > 3 , remove all nodes with degree $\leq 3 - 1 = 2$
 - Remove nodes 2 and 9
 - Remove nodes 1 and 3
 - Remove node 4



- Clique is a very strict definition, unstable
- Normally use cliques as a core or a seed to find larger communities
- CPM is such a method to find overlapping communities
- Input:
 - Parameter k and the network
- Procedure
 - Given a network, find out all cliques of size k
 - Construct a clique graph. Two cliques are adjacent if they share $k - 1$ nodes
 - Each connected components in the clique graph form a community

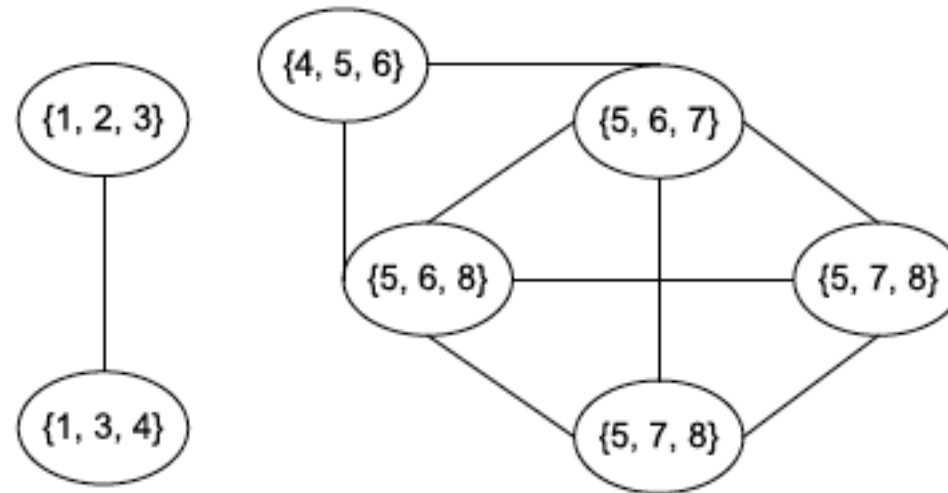


Cliques of size 3:

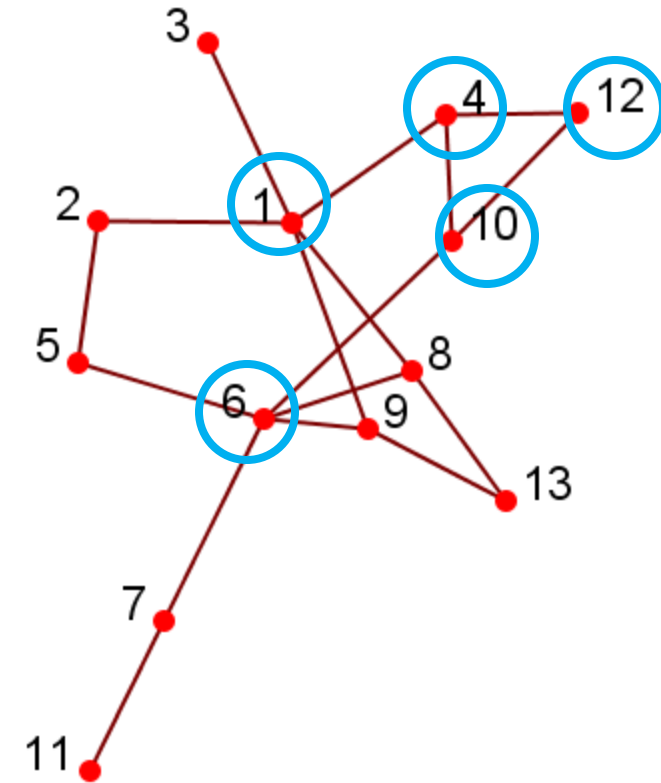
$\{1, 2, 3\}$, $\{1, 3, 4\}$, $\{4, 5, 6\}$, $\{5, 6, 7\}$,
 $\{5, 6, 8\}$, $\{5, 7, 8\}$, $\{6, 7, 8\}$

Communities:

+ $\{1, 2, 3, 4\}$
 + $\{4, 5, 6, 7, 8\}$



- Any node in a group should be reachable in k hops
- k -clique: a maximal subgraph in which the largest geodesic distance between any nodes $\leq k$
- A k -clique can have diameter larger than k within the subgraph
 - e.g., 2-clique: {12, 4, 10, 1, 6}. In this clique, the distance between any two nodes ≤ 2 -hop
- k -club: a substructure of diameter $\leq k$
 - It means that every node pair is connected by at least one path with at most k edges
 - e.g., {1, 2, 5, 6, 8, 9}, {12, 4, 10, 1} are 2-clubs



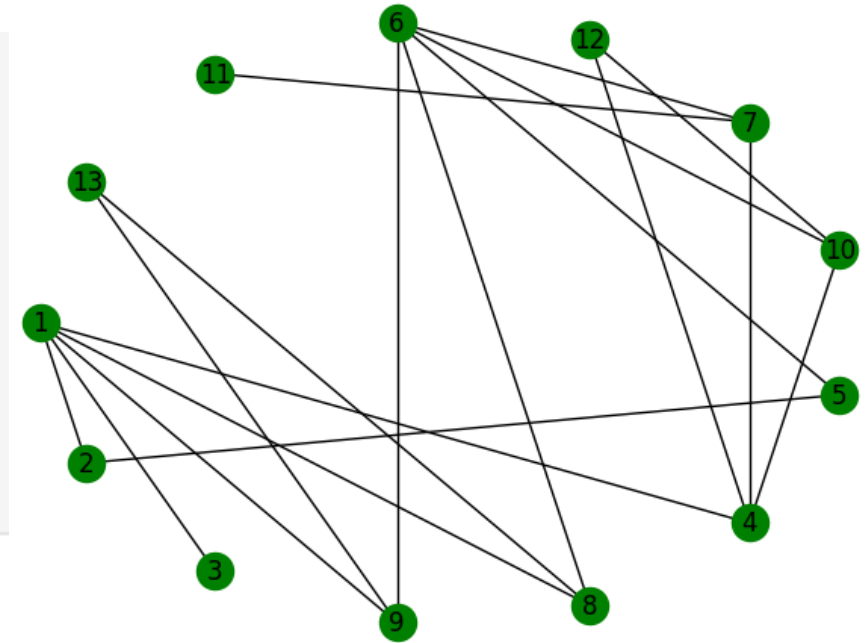
- Find k-clique communities in graph using the percolation method:

```
from networkx.algorithms.community import k_clique_communities

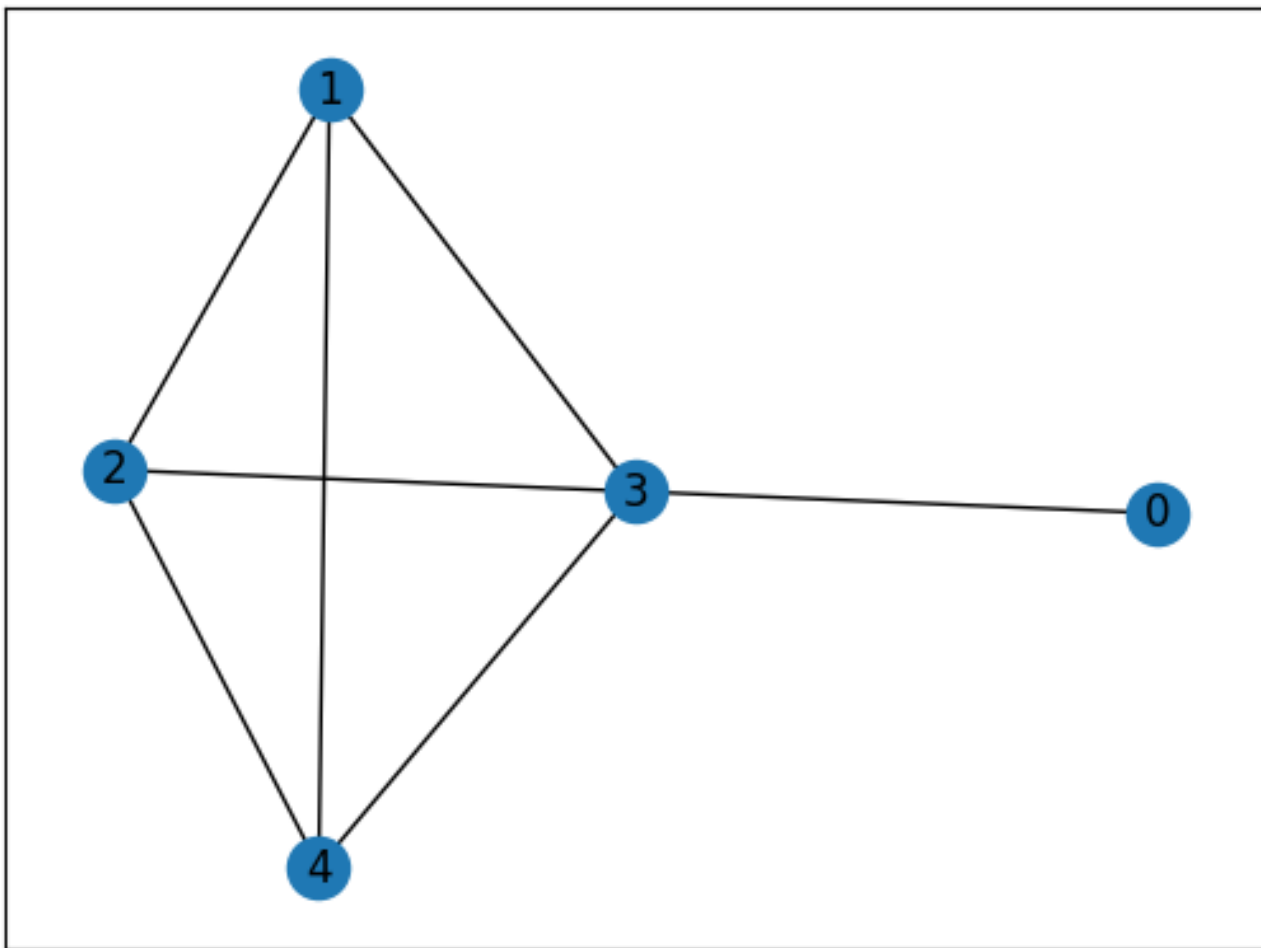
G = nx.Graph()
edges = [(1, 2), (1, 3), (1, 9), (1, 8), (1, 4), (2, 5), (4, 10), (4, 7),
         (4, 12), (5, 6), (6, 7), (6, 8), (6, 9), (6, 10), (7, 11), (8, 13), (9, 13), (10, 12)]

G.add_edges_from(edges)
nx.draw(G, node_color="green", with_labels=True, pos=nx.shell_layout(G))
c = list(k_clique_communities(G, 3))
sorted(list(c[0]))
```

```
[4, 10, 12]
```



```
: import networkx as nx
G1 = nx.Graph()
edges = [(1, 2), (2, 3), (1, 3), (3, 4), (3, 0), (1, 4), (4, 2)]
G1.add_edges_from(edges)
nx.draw_networkx(G1)
```



```
res = nx.find_cliques(G1)
cliques = [item for item in res]
cliques = sorted(cliques, key=lambda item: -len(item))
for item in cliques:
    print(item)
```

```
[3, 1, 2, 4]
[3, 0]
```

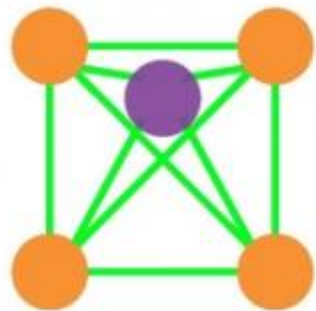

- The clustering coefficient measures how connected a vertex's neighbours are to one another
- The range is from 0 to 1 (from non-neighbour are connected to each other to all neighbours are fully connected)
- There are three types of clustering coefficient:
 - Local clustering coefficient
 - Average clustering coefficient
 - Global clustering coefficient

- How close its neighbours are to being a clique (complete graph)
- For a node i with degree d_i and L_i represents the number of edges between neighbors of node i

The local clustering coefficient C_i for a node i is defined as:

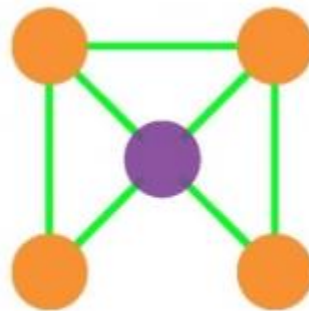
$$C_i = \frac{2L_i}{d_i(d_i - 1)}$$

neighbors of node i form a complete graph

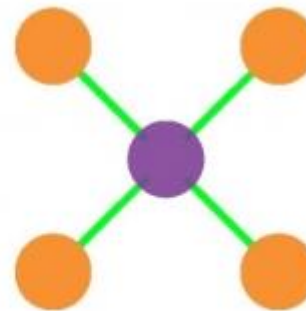


$$C_i = 1$$

50% chance that two neighbors of a node i are linked



$$C_i = 1/2$$

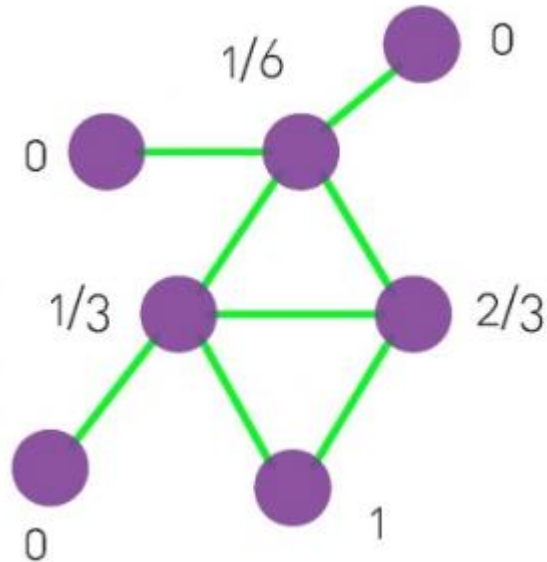


$$C_i = 0$$

None of neighbors of node i link to each other

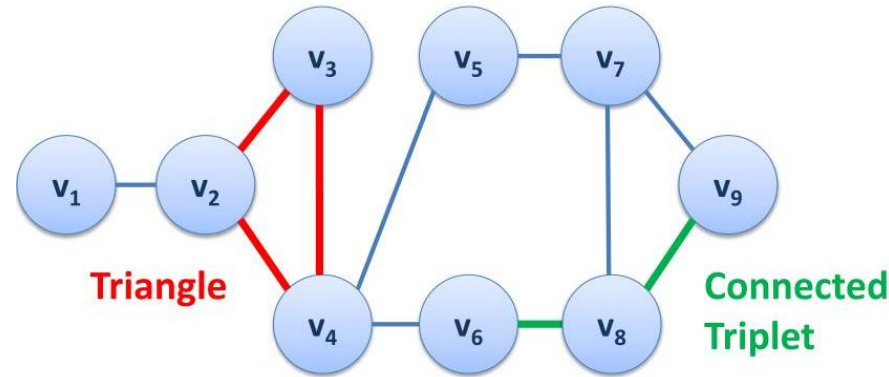
- The degree of clustering of a whole network is captured by the average clustering coefficient, namely $\langle C \rangle$, representing the average of all the local clustering coefficient C_i over all nodes $i = 1, \dots, N$

$$\langle C \rangle = \frac{1}{N} \sum_{i=0}^N C_i$$



$$\langle C \rangle = \frac{1}{7} * \left(0 + \frac{1}{6} + \frac{1}{3} + \frac{2}{3} + 1 + 0 + 0 \right) = 0.333$$

- The global clustering coefficient is based on triplets of nodes
- A triplet consists of three connected nodes. A triangle therefore includes three closed triplets, one centered on each of the nodes

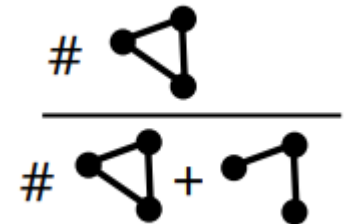


- The global clustering coefficient is the number of closed triplets over the total number of triplets (both open and closed)

Closed triplets: $(v_2, v_3, v_4), (v_7, v_8, v_9)$

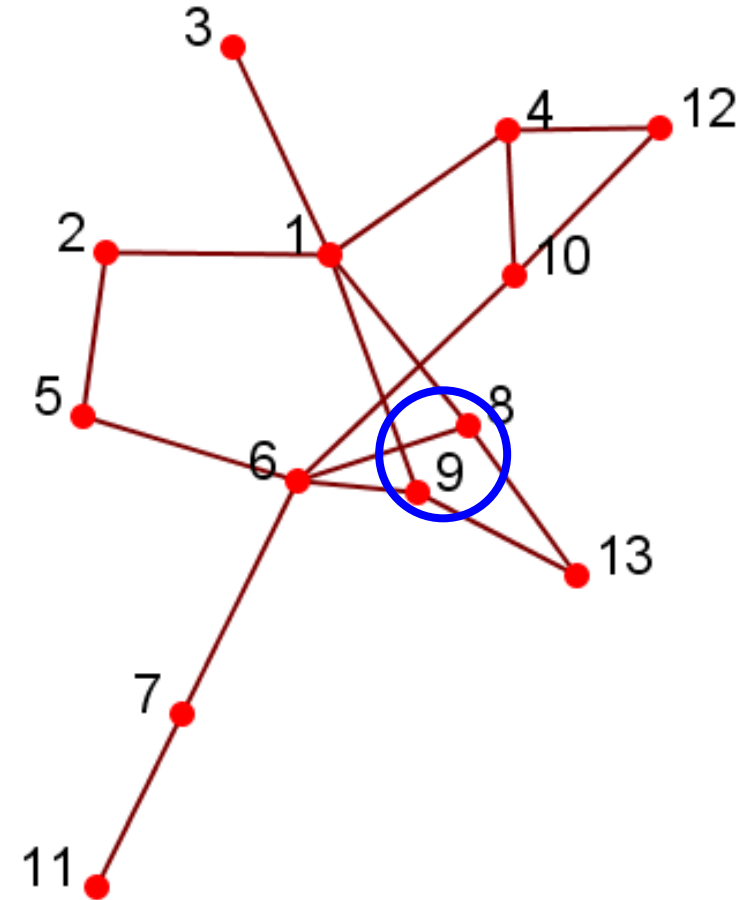
Connected triplets: $(v_6, v_8, v_9), \dots$

$$C(G) = \frac{\# \text{ of closed triplets}}{\# \text{ of connected triplets}}$$



- To form a group, we need to consider the connections of the nodes globally
- **Goal:** partition the network into disjoint sets
- Groups based on:
 - Node Similarity
 - Cut Minimization
 - Louvain

- Node similarity is defined by how similar their interaction patterns are
- Two nodes are structurally equivalent if they connect to the same set of actors
 - e.g., nodes 8 and 9 are structurally equivalent
- Groups are defined over equivalent nodes:
 - Too strict
 - Rarely occur in a large-scale
 - Relaxed equivalence class is difficult to compute
- In practice, use vector similarity
 - e.g., cosine similarity, Jaccard similarity



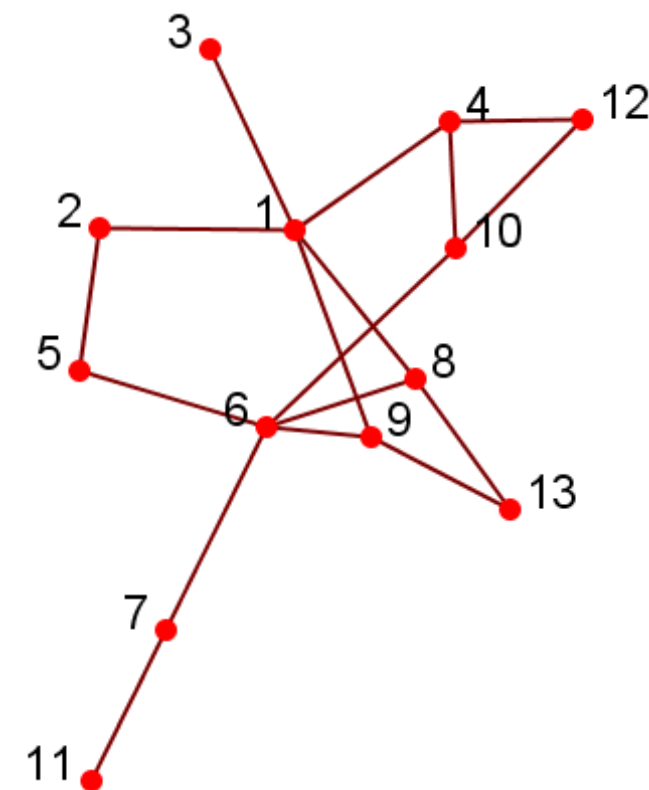
		1	2	3	4	5	6	7	8	9	10	11	12	13
A vector →	5		1				1							
Structurally equivalent {	8	1					1							1
	9	1					1							1

Cosine Similarity: $\text{similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$

$$\text{sim}(5,8) = \frac{1}{\sqrt{2} \times \sqrt{3}} = \frac{1}{\sqrt{6}}$$

Jaccard Similarity: $J(A,B) = \frac{|A \cap B|}{|A \cup B|}$

$$J(5,8) = \frac{|\{6\}|}{|\{1,2,6,13\}|} = 1/4$$

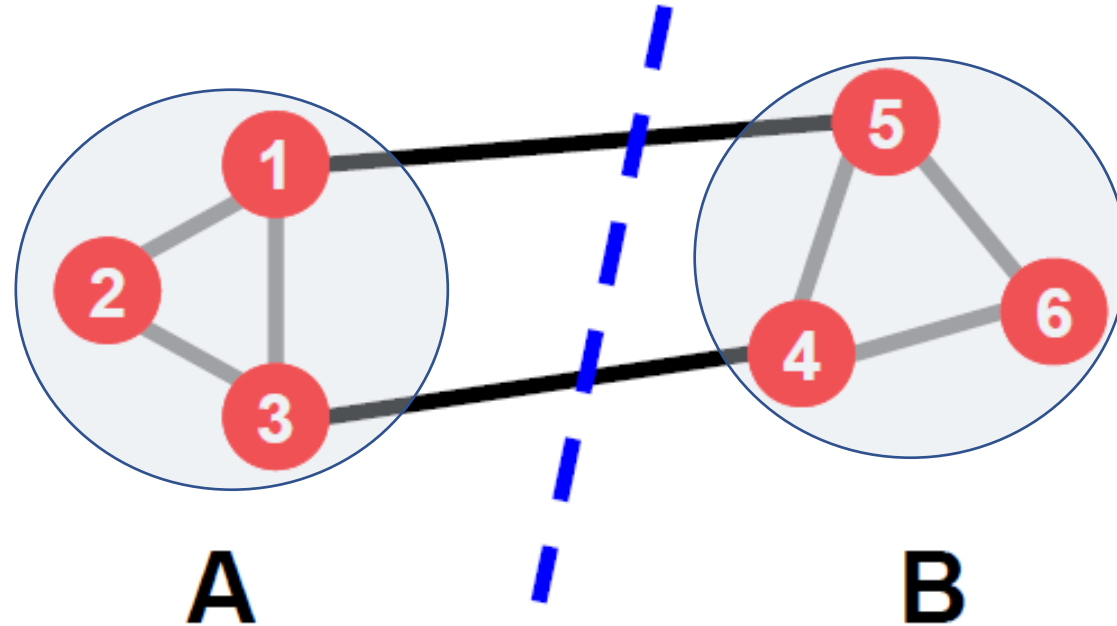


- For practical use with huge networks:
 - Consider the connections as features
 - Use Cosine or Jaccard similarity to compute vertex similarity
 - Apply classical k-means clustering Algorithm
- K-means Clustering Algorithm:
 - Each cluster is associated with a centroid (centre point)
 - Each node is assigned to the cluster with the closest centroid

Algorithm 1 Basic K-means Algorithm.

- 1: Select K points as the initial centroids.
 - 2: **repeat**
 - 3: Form K clusters by assigning all points to the closest centroid.
 - 4: Recompute the centroid of each cluster.
 - 5: **until** The centroids don't change
-

- Basic principle for graph partitioning:
 - Minimize the number of between-group connections
 - Maximize the number of within-group connections

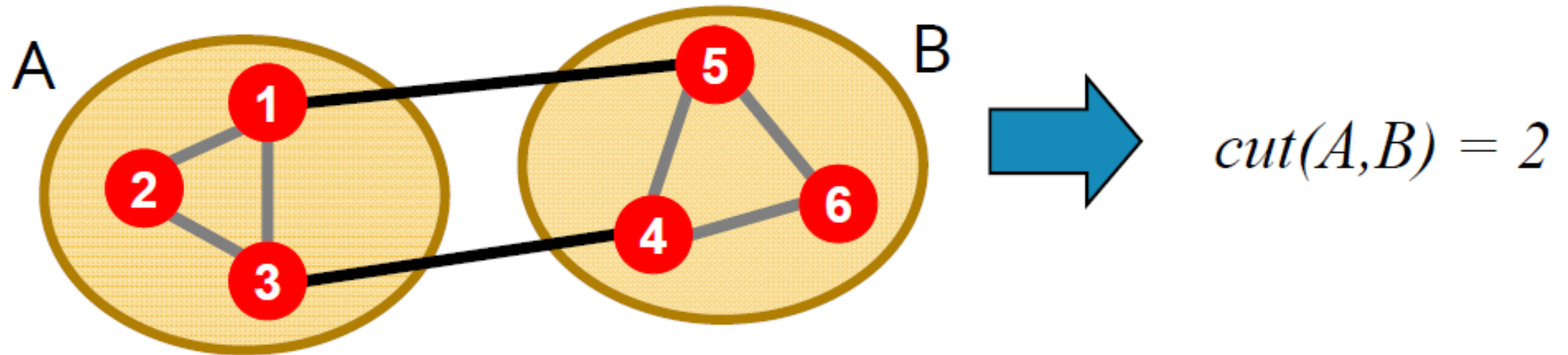


Graph partitioning : A & B

- Basic principle for graph partitioning:
 - Minimize the number of between-group connections
 - Maximize the number of within-group connections

	Min-cut	N-cut
Minimize: between group connections	✓	✓
Maximize : within-group connections	X	✓

- For considering between-group:



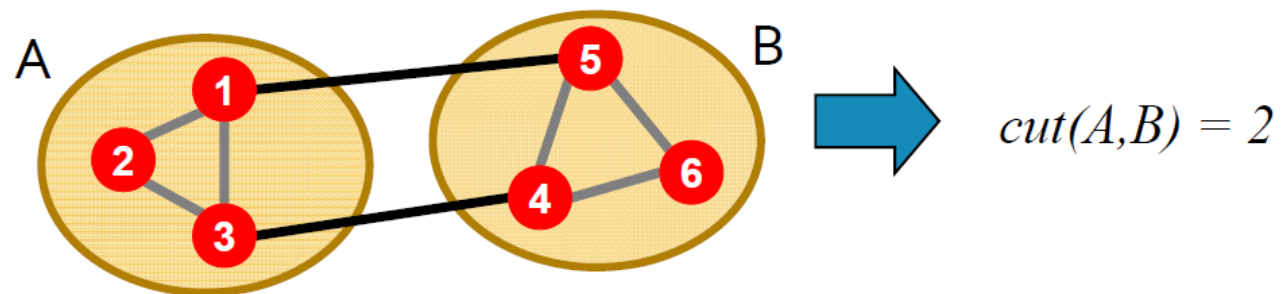
Cut: Set of edges with only one vertex in a group:

$$cut(A,B) = \sum_{i \in A, j \in B} w_{ij}$$

- For considering within-group
- $\text{Assoc}(A, V)$: the total connection from nodes in A to all nodes in the graph

$$\text{assoc}(A, V) = \sum_{u \in A, t \in V} w(u, t)$$

$$\text{Ncut}(A, B) = \frac{\text{cut}(A, B)}{\text{assoc}(A, V)} + \frac{\text{cut}(A, B)}{\text{assoc}(B, V)}$$

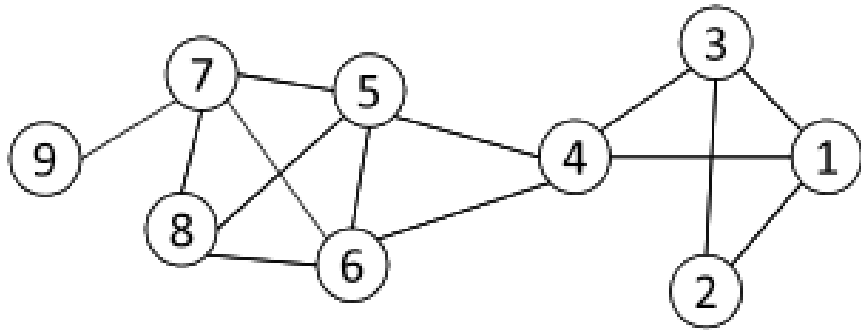


- Goal: build a hierarchical structure of communities based on network topology.
- Allow the analysis of a network at different resolutions.
- Representative approaches:
 - Divisive Hierarchical Clustering.
 - Agglomerative Hierarchical Clustering.

- Divisive clustering:
 - Partition nodes into several sets
 - Each set is further divided into smaller ones
 - Network-centric partition can be applied for the partition
- One example: recursively remove the “weakest” tie:
 - Find the edge with the least strength
 - Remove the edge and update the corresponding strength of each edge
- Recursively apply the above two steps until a network is decomposed into desired number of connected components
- Each component forms a community

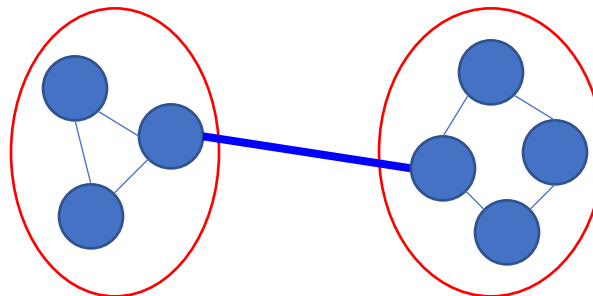
- The strength of a tie can be measured by edge betweenness
- Edge betweenness: the number of shortest paths that pass along with the edge

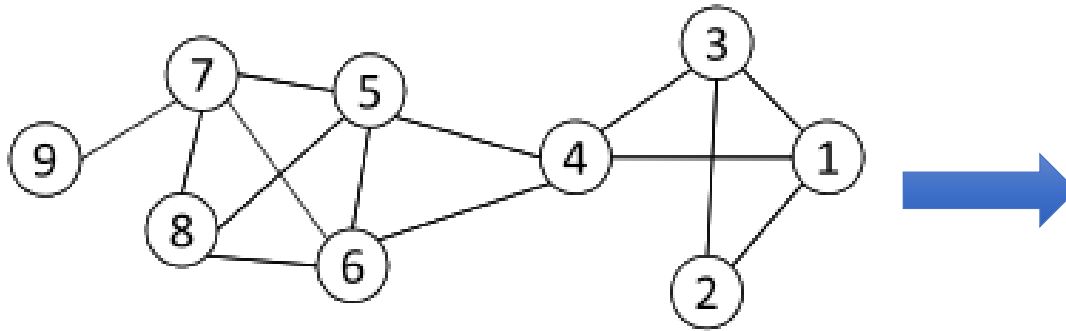
$$\text{edge-betweenness}(e) = \sum_{s < t} \frac{\sigma_{st}(e)}{\sigma_{s,t}}$$



The edge betweenness of $e(1, 2)$ is 4, as all the shortest paths from 2 to $\{4, 5, 6, 7, 8, 9\}$ have to either pass $e(1, 2)$ or $e(2, 3)$, and $e(1,2)$ is the shortest path between 1 and 2

- The edge with higher betweenness tends to be the bridge between two communities

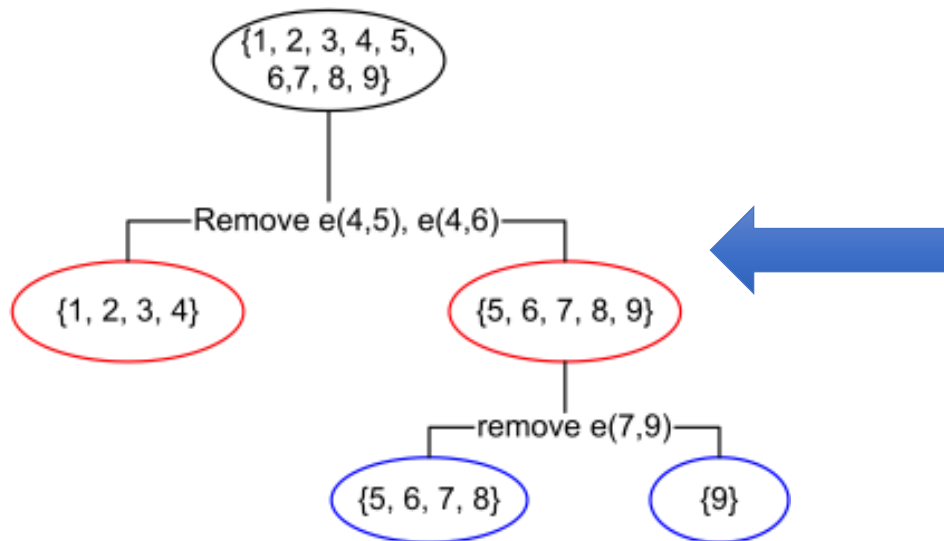




Initial betweenness value

Table 3.3: Edge Betweenness

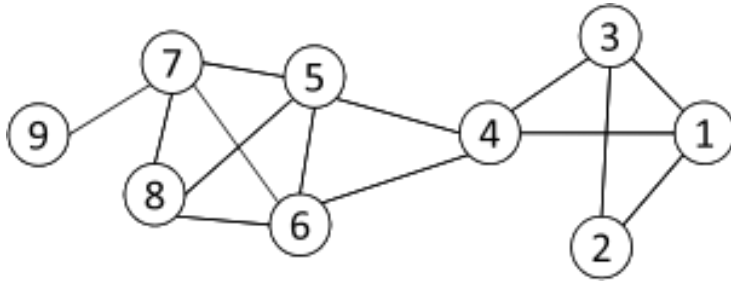
	1	2	3	4	5	6	7	8	9
1	0	4	1	9	0	0	0	0	0
2	4	0	4	0	0	0	0	0	0
3	1	4	0	9	0	0	0	0	0
4	9	0	9	0	10	10	0	0	0
5	0	0	0	10	0	1	6	3	0
6	0	0	0	10	1	0	6	3	0
7	0	0	0	0	6	6	0	2	8
8	0	0	0	0	3	3	2	0	0
9	0	0	0	0	0	0	8	0	0



- After remove $e(4,5)$, the betweenness of $e(4, 6)$ becomes 20, which is the highest.

- After remove $e(4,6)$, the edge $e(7,9)$ has the highest betweenness value 4 and should be removed.

- Modularity measures the strength of a community partition by considering the degree distribution
- Given a graph with m edges, the expected number of edges between two nodes with degree d_i and d_j is $d_i d_j / 2m$



The expected number of edges between nodes 1 and 2 is $3 \cdot 2 / (2 \cdot 14) = 3/14$

- Modularity:

$$Q = \frac{1}{2m} \sum_{ij} (A_{ij} - \gamma \frac{d_i d_j}{2m}) \delta(C_i, C_j)$$

probability a random edge would go between i and j

Adjacency matrix of graph

$\delta(C_i, C_j)$: 1 if i and j are in the same community else 0

- We have modularity matrix:

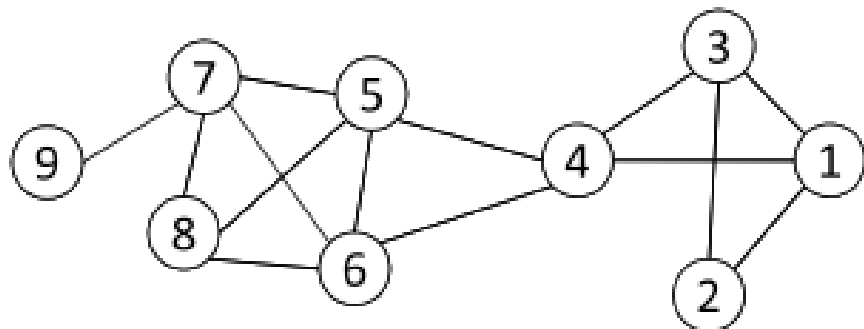
$$B = A - \mathbf{d}\mathbf{d}^T / 2m \quad (B_{ij} = A_{ij} - d_i d_j / 2m)$$

Where: d is node degree

- Similar to spectral clustering, modularity maximization can be reformulated as

$$\max Q = \frac{1}{2m} \text{Tr}(S^T B S) \quad s.t. \quad S^T S = I_k$$

- Optimal solution: top eigenvectors of the modularity matrix
- Apply k-means to S as a post-processing step to obtain community partition



Two Communities:
 $\{1, 2, 3, 4\}$ and $\{5, 6, 7, 8, 9\}$

$\uparrow$
 k-means

$$B = \begin{bmatrix} -0.32 & 0.79 & 0.68 & 0.57 & -0.43 & -0.43 & -0.43 & -0.32 & -0.11 \\ 0.79 & -0.14 & 0.79 & -0.29 & -0.29 & -0.29 & -0.29 & -0.21 & -0.07 \\ 0.68 & 0.79 & -0.32 & 0.57 & -0.43 & -0.43 & -0.43 & -0.32 & -0.11 \\ 0.57 & -0.29 & 0.57 & -0.57 & 0.43 & 0.43 & -0.57 & -0.43 & -0.14 \\ -0.43 & -0.29 & -0.43 & 0.43 & -0.57 & 0.43 & 0.43 & 0.57 & -0.14 \\ -0.43 & -0.29 & -0.43 & 0.43 & 0.43 & -0.57 & 0.43 & 0.57 & -0.14 \\ -0.43 & -0.29 & -0.43 & -0.57 & 0.43 & 0.43 & -0.57 & 0.57 & 0.86 \\ -0.32 & -0.21 & -0.32 & -0.43 & 0.57 & 0.57 & 0.57 & -0.32 & -0.11 \\ -0.11 & -0.07 & -0.11 & -0.14 & -0.14 & -0.14 & 0.86 & -0.11 & -0.04 \end{bmatrix}$$

Modularity Matrix



$$S = \begin{bmatrix} 0.44 & -0.00 \\ 0.38 & 0.23 \\ 0.44 & -0.00 \\ 0.17 & -0.48 \\ -0.29 & -0.32 \\ -0.29 & -0.32 \\ -0.38 & 0.34 \\ -0.34 & -0.08 \\ -0.14 & 0.63 \end{bmatrix}$$

```
def modularity(G, partition):
    W = sum(G.edges[v, w].get('weight', 1) for v, w in G.edges)
    summation = 0
    for cluster_nodes in partition:
        s_c = sum(G.degree(n, weight='weight') for n in cluster_nodes)
        # Use subgraph to count only internal links
        C = G.subgraph(cluster_nodes)
        W_c = sum(C.edges[v, w].get('weight', 1) for v, w in C.edges)
        summation += W_c - s_c ** 2 / (4 * W)

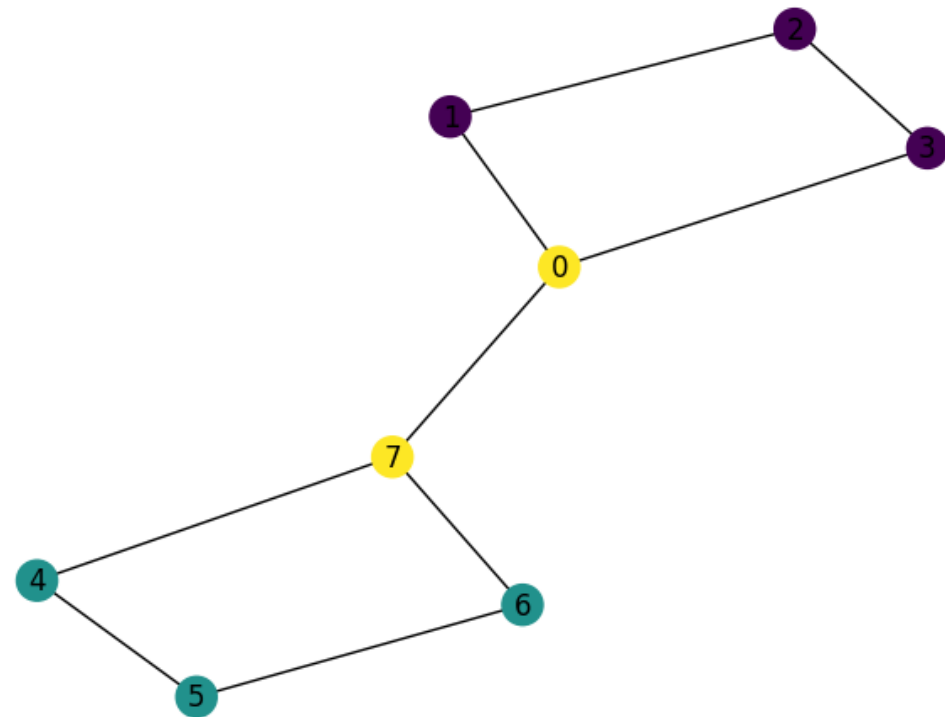
    return summation / W
```

```
modularity(G, partition)
```

```
0.2222222222222222
```

```
|: G = nx.Graph()
   nx.add_cycle(G, [0, 1, 2, 3])
   nx.add_cycle(G, [4, 5, 6, 7])
   G.add_edge(0, 7)

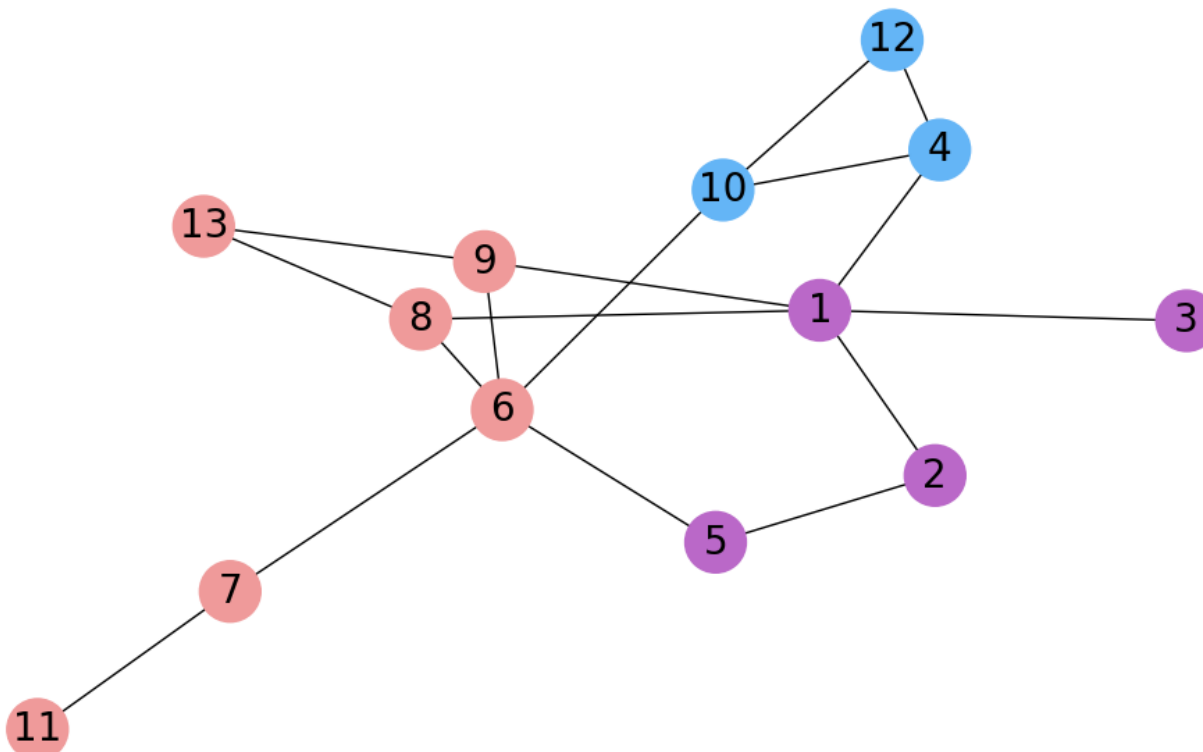
   nx.draw(G, with_labels=True)
```



- Louvain algorithm is an efficient hierarchical clustering algorithm based on graph theory
- Its principle is to make the modularity of community partition result reach the maximum value through continuous iteration of mobile nodes and then obtain the optimal community partition

- Step 1: Initialize the community and set each node as a separate community, namely, community 1: (node1), community 2: (node2), and so on
- Step 2: Find out all the communities connected to node 1 and calculate the change of modularity after moving node 2 to each neighbour community. Move node 1 to the community, which can increase the modularity to the maximum
- Step 3: Iterate over all the nodes and execute step 2 until there are no nodes to move and get a layer of community partition
- Step 4: Merge each community in step 3 into a new node. The relationship between new nodes is the relationship between the original communities. Return to step 1 until all nodes are finally merged into one community

```
# convert the python-louvain package output to NetworkX package community function output format
def get_louvain_communities(graph, random_state=1):
    louvain_partition_dict = community_louvain.best_partition(graph, random_state=random_state)
    unique_partition_labels = list(set(louvain_partition_dict.values()))
    communities = [[] for i in range(len(unique_partition_labels))]
    for node in louvain_partition_dict.keys():
        communities[louvain_partition_dict[node]].append(node)
    return communities
```



- Louvain community detection tends to discover communities that are internally disconnected
 - A node moving from a community to a new community may disconnect the old community
- Leiden community detection is not only fast and efficient, but also guarantees that communities are well connected

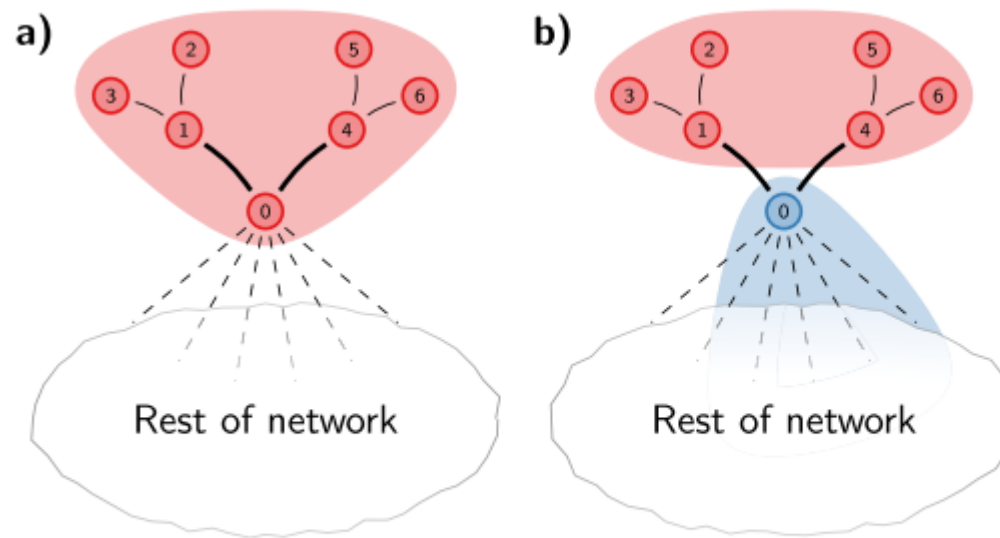
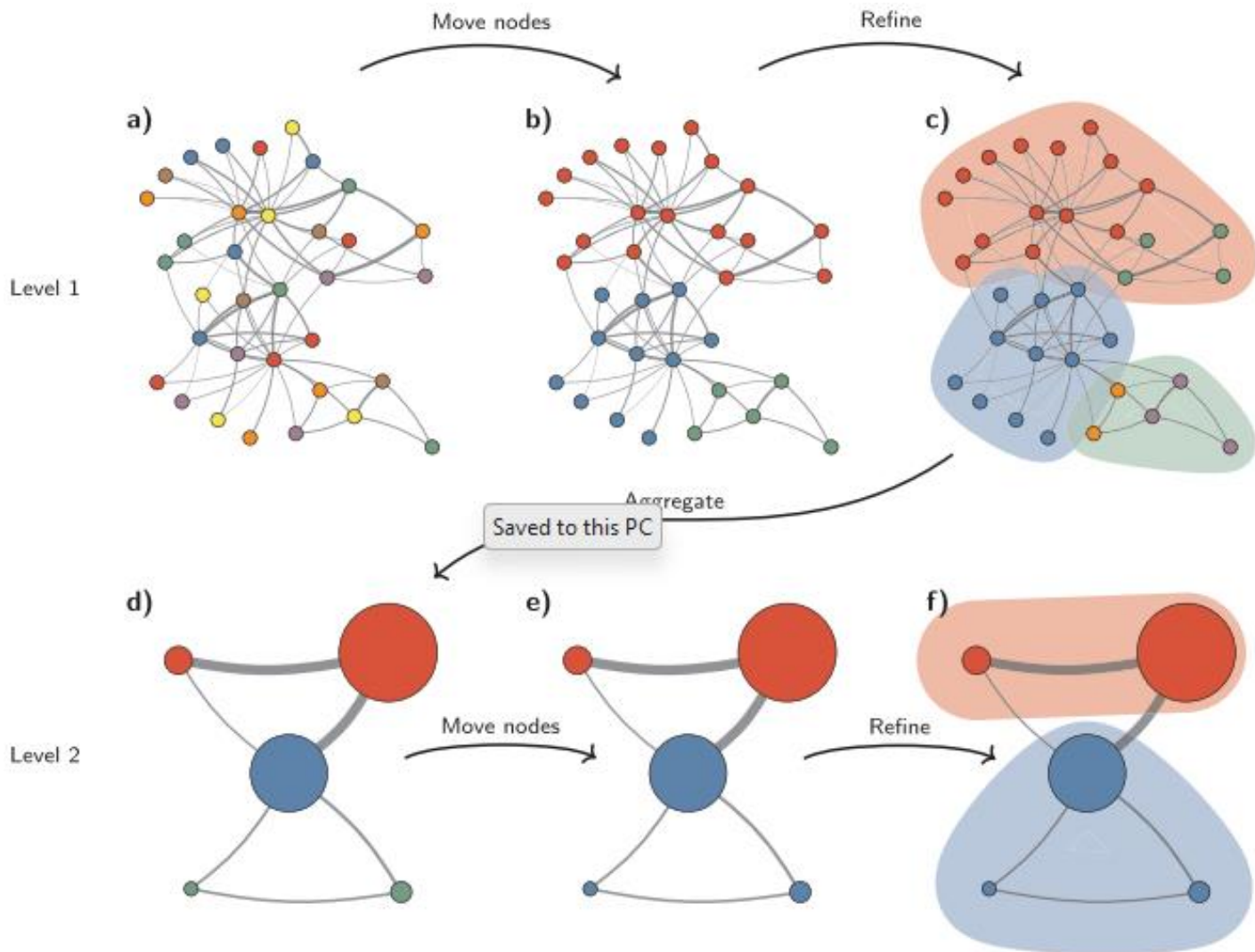


Figure 2. Disconnected community. Consider the partition shown in (a). When node 0 is moved to a different community, the red community becomes internally disconnected, as shown in (b). However, nodes 1–6 are still locally optimally assigned, and therefore these nodes will stay in the red community.

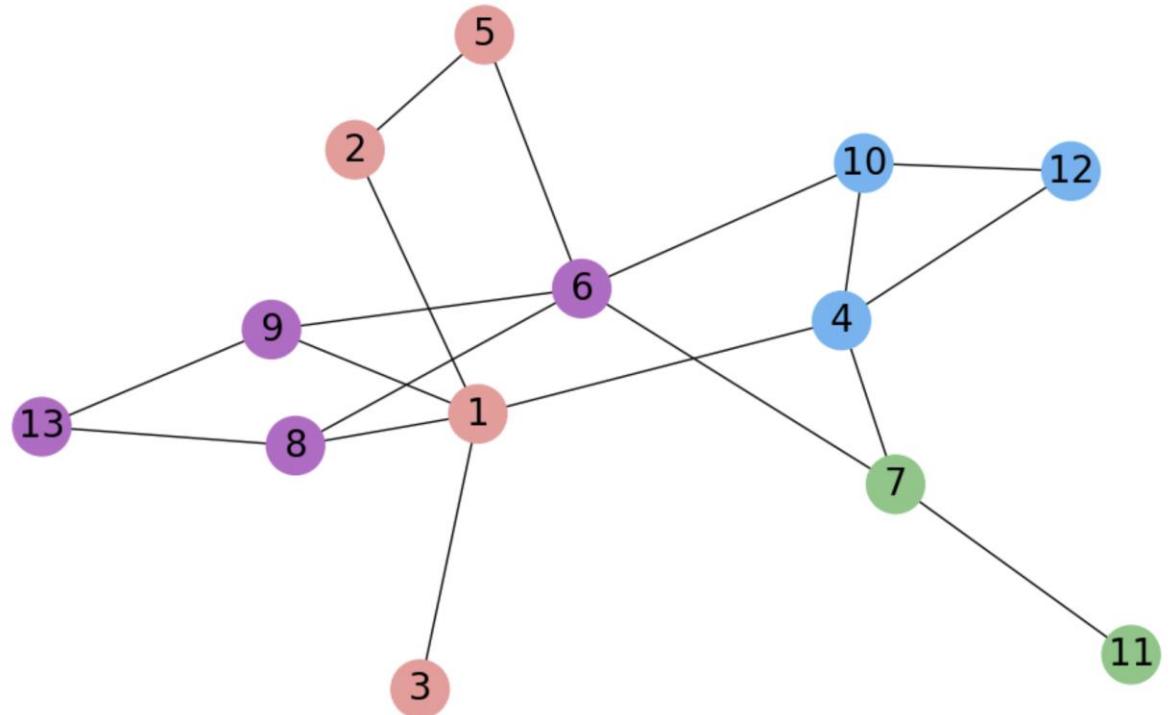
- Three main phases:
 - Local moving of nodes (a – b)
 - Refinement of the partitions (b- c)
 - Aggregation of the network based on refined partitions (c-d)
 - Repeat until no further improvements (d-f)



```
# convert the leiden package output to NetworkX package community function output format and visualize
def get_visualize_leiden_communities(graph, random_state=1):
    leiden_communities = leiden(G).communities
    node_colors = create_community_node_colors(graph, leiden_communities)
    modularity = round(nx_comm.modularity(graph, leiden_communities), 6)
    print("Leiden communities",leiden_communities)
    title = f"Community Visualization of {len(leiden_communities)} communities with modularity of {modularity}"
    pos = nx.spring_layout(graph,
                           k=0.3, iterations=50,
                           seed=2)

    plt.figure(1,figsize=(10,6))
    nx.draw(graph,
            pos = pos,
            node_size=1000,
            node_color=node_colors,
            with_labels=True,
            font_size = 20,
            font_color='black')
    plt.title(title)
    plt.show()
```

Community Visualization of 4 communities with modularity of 0.328704



- The Shift to Deep Learning: Traditional statistical models (like SBM) struggle with high-dimensional data and complex node attributes.
- Core Advantage: Deep learning maps network data from a high-dimensional input space to a low-dimensional feature space (embeddings).
- Key Capabilities:
 - Effective data fusion (topology + attributes).
 - Low computational complexity.
 - High capability for parallelization.

- Deep learning methods for community detection are categorized into four primary streams:
 1. **Auto-encoder-based**: Unsupervised learning of latent representations.
 2. **Generative Adversarial Network (GAN)-based**: Adversarial games to fit data distributions.
 3. **Message Passing Neural Network (MPNN)-based**: Propagation and aggregation of local features.
 4. **Hybrid Models**: Integrating GCNs with Undirected Graphical Models (like MRF) to combine global and local strengths.

Auto-encoder-based Methods

Auto-encoders compress network data into a hidden representation to minimize reconstruction error.

- **Stacked Auto-encoders:** Use multiple layers to extract salient features (e.g., Semi-DNR).
- **Sparse Auto-encoders:** Add sparsity constraints to handle large-scale networks (e.g., GraphEncoder).
- **Denoising Auto-encoders:** Robust to noisy inputs by reconstructing data from corrupted versions (e.g., MGAE).
- **Variational Auto-encoders (VAE):** Treat hidden representations as latent variables with prior distributions (e.g., VGAECD).

Auto-encoder-based Methods

The objective of a basic auto-encoder is to minimize the reconstruction error between the original input and the data reconstructed by the decoder.

$$Loss(\theta_1, \theta_2) = \sum_{i=1}^n l(x_i, g(f(x_i; \theta_1); \theta_2))$$

- $f(\cdot; \theta_1)$: The encoder that maps input x to a lower-dimensional latent space.
- $g(\cdot; \theta_2)$: The decoder that attempts to reconstruct the original data from the latent representation.
- $l(\cdot)$: The loss function (often Mean Squared Error or Cross-Entropy).

For Variational Auto-encoders (VAE), the loss includes a KL-divergence term to regularize the latent space:

$$Loss = \mathbb{E}_{q(H|(X, A))} [\log p(\hat{A}|H)] - KL[q(H|X, A) || p(H)]$$

Auto-encoder-based Methods

Categories	Approaches	A	X	Encoder	Decoder	Focus	Constraints
Stacked	Semi-DNR (2016)	Yes	No	MLP	MLP	Network embedding	Pairwise constraint
	DIR (2017)	Yes	Yes	MLP	MLP	Network embedding	-
	INSNCD (2018)	Yes	Yes	MLP	MLP	Network embedding	Modularity maximization
	AAGR (2018)	Yes	Yes	MLP	MLP	Network embedding	Adaptive parameter
	CDDTA (2019)	Yes	No	MLP	MLP	Network embedding	Regularization term
	DeCom (2019)	Yes	No	MLP	MLP	Clustering result	Modularity maximization
	NEC (2020)	Yes	Yes	GCN	Inner product	Embedding and clustering	Modularity maximization

Summary of auto-encoder-based community detection, where "A" and "X" denote whether the approach utilize network topology and node attributes respectively, and "-" represents no constraint

Auto-encoder-based Methods

Categories	Approaches	A	X	Encoder	Decoder	Focus	Constraints
Sparse	GraphEncoder (2014)	Yes	No	MLP	MLP	Network embedding	Sparsity constraint
	DFuzzy (2018)	Yes	No	MLP	MLP	Clustering result	Sparsity constraint and modularity maximization
	CDMEC (2020)	Yes	No	MLP	MLP	Clustering result	Sparsity constraint
Denoising	MGAE (2017)	Yes	Yes	GCN	GCN	Network embedding	Interplay exploitation
	GRACE (2017)	Yes	Yes	MLP	MLP	Embedding and clustering	Propagation constraint

Summary of auto-encoder-based community detection, where "A" and "X" denote whether the approach utilize network topology and node attributes respectively, and "-" represents no constraint

Auto-encoder-based Methods

Categories	Approaches	A	X	Encoder	Decoder	Focus	Constraints
Variational	ARVGA (2018)	Yes	Yes	GCN	Inner product	Network embedding	Prior constraint
	VGAECD (2018)	Yes	Yes	GCN	Inner product	Embedding and clustering	-
	DAEGC (2019)	Yes	Yes	GAT	Inner product	Embedding and clustering	KL divergence constraint
	New VGAECD (2019)	Yes	Yes	GCN	Inner product	Embedding and clustering	-
	Ladder VAE (2019)	Yes	No	GCN	MLP	Embedding and clustering	-
	NetVAE (2019)	Yes	Yes	MLP	MLP	Network embedding	Prior constraint

Summary of auto-encoder-based community detection, where "A" and "X" denote whether the approach utilize network topology and node attributes respectively, and "-" represents no constraint

Generative Adversarial Network (GAN) Methods

GANs use a Generator (G) and a Discriminator (D) in a minimax game.

Concept: G tries to generate realistic community structures, while D tries to distinguish between real and synthetic ones.

Key Example: CommunityGAN:

- Jointly solves overlapping community detection and graph representation learning.
- Represents the strength of membership of a node to a community.

Advantage: GANs excel at learning the underlying representation mechanism rather than just static embeddings.

Generative Adversarial Network (GAN) Methods

GAN-based community detection relies on a minimax game between a generator (G) and a discriminator (D).

$$\min_G \max_D V(G, D) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log (1 - D(G(z)))]$$

- **The Players:** It is modeled as a two-player game between a Generator (G) and a Discriminator (D).
- **Discriminator's Goal (max D):** D tries to maximize the probability of correctly identifying real data (x) from the true data distribution (p_{data}) and identifying fake data generated by G .
- **Generator's Goal (min G):** G tries to minimize the probability that D identifies its samples as fake by generating data as similar to the real distribution as possible.
- **Adversarial Logic:** In community detection, this allows the model to learn the underlying structural representation of a network by trying to "fake" realistic community memberships.

Generative Adversarial Network (GAN) Methods

In the specific case of **CommunityGAN**, the value function is formulated to handle node-community memberships:

$$\min_{\theta_G} \max_{\theta_D} V(G, D) = \sum_{i=1}^n \left(\mathbb{E}_{\mathbf{m} \sim \mathbb{P}_{\text{true}}(\cdot | \mathbf{v}_i)} [\log D(\mathbf{m}; \theta_D)] + \mathbb{E}_{\mathbf{s} \sim \mathbb{G}(\mathbf{s} | \mathbf{v}_i; \theta_G)} \left[\log \left(1 - D(G(\mathbf{s}; \theta_D)) \right) \right] \right)$$

- **Node-Community Memberships:** Instead of general data points, the parameters (θ_G and θ_D) represent vectors of all nodes, where each entry represents the degree of membership of a node to a specific community.
- **Motifs (m):** The discriminator (D) is trained on network motifs—small, recurring sub-graphs that represent "true" local community structures.
- **Subsets (s):** The generator (G) constructs subsets of nodes to trick the discriminator into believing they are valid community motifs.
- **Summation ($\sum_{i=1}^n$):** The loss is calculated across all n nodes in the network to ensure the learned embeddings capture the global community structure.

Message Passing Neural Networks (MPNN)

MPNNs are the most representative branch for learning directly from graph data.

- **Mechanism:** They use local filters to aggregate information from neighboring nodes.
- **Unsupervised MPNN:** Since community detection is unsupervised, models like JGE-CD use dual encoders to reconstruct both topology and attributes.
- **Recent Trend:** Application to Heterogeneous Networks (multiple node/edge types) and Temporal Networks (evolving over time).

Message Passing Neural Networks (MPNN)

In unsupervised community detection using MPNNs, the probability of node v_i belonging to community r is often defined using a softmax-like function over the learned embeddings:

$$i_{ir} = \frac{\exp(\theta_r^T h_i)}{\sum_{j=1}^k \exp(\theta_j^T h_i)}$$

- h_i : The embedding of node v_i obtained from the MP layers.
- θ_r : The trainable model parameters associated with community r .

Hybrid Models: MPNN + Undirected Models

This approach integrates the local smoothing of MPNNs with the global objectives of Markov Random Fields (MRF).

- **The Logic:** MPNNs capture node features well but aren't always community-oriented; MRFs offer a global objective for communities but ignore node features.
- **Featured Model: MRFasGCN:**
 - Combines unary potentials (from GCN) with pairwise potentials (from MRF).
 - Optimizes parameters in a single end-to-end architecture.

Hybrid Models: MPNN + Undirected Models

Models like MRFasGCN integrate local feature smoothing with global community constraints using an energy function:

$$E(C; A, X) = \sum_{v_i} -p(h_i^{c_i}) + \alpha \sum_{v_i \neq v_j} \mu(c_i, c_j) \eta(v_i, v_j)$$

- $-p(h_i^{c_i})$: The unary potential (derived from GCN output) representing the probability of node v_i belonging to community c_i .
- $\mu(c_i, c_j) \eta(v_i, v_j)$: The pairwise potential representing the similarity between communities and node attributes.
- α : A trade-off parameter balancing the two potentials.

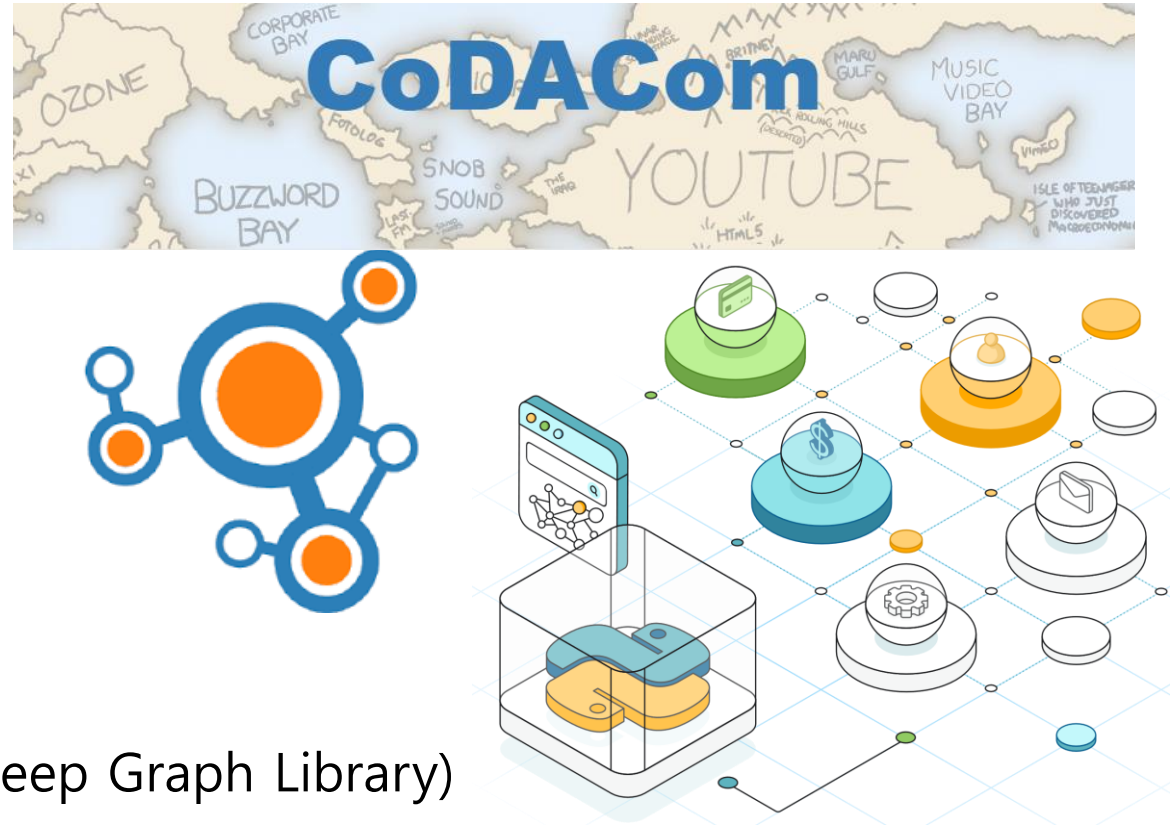
➤ Algorithms:

- Label Propagation
- InfoMap
- Girvan-Newman
- Surprise Community Detection
- Etc.

➤ Library (Python):

- NetworkX
- CoDACom
- CyCommunityDetection
- PyTorch Geometric (PyG)

- DGL (Deep Graph Library)
- SNAP (Stanford)
- Neo4j Graph Data Science
- Etc.



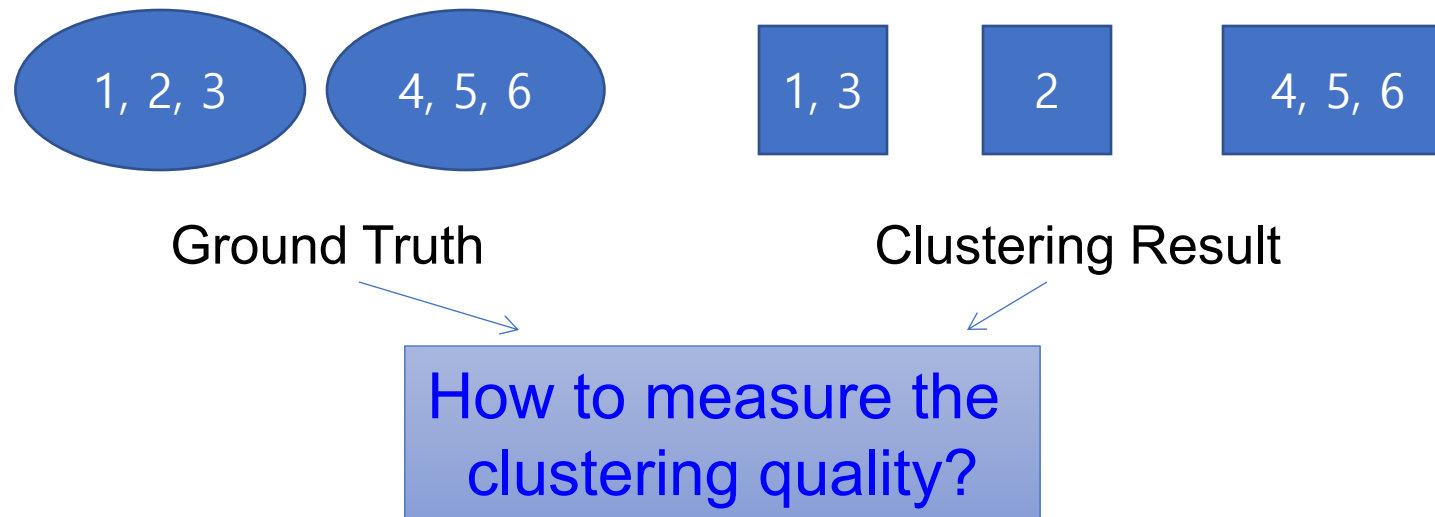
Category	Tool	Method
GNN	PyTorch	AGE
		LGNN
		NOCD
	TensorFlow	AGC
		CayleyNet
Attentive GNN	PyTorch	CP-GNN
		DMGI
		MAGNN
		HDMI
		HeCo
GAN	PyTorch	SEAL
	TensorFlow	CommunityGAN
Stacked AE	TensorFlow	DANE
	Matlab	DIME
		DNE-SBP
		semi-DRN

Source: Su, Xing, et al. "A comprehensive survey on community detection with deep learning." *IEEE transactions on neural networks and learning systems* 35.4 (2022): 4682-4702.

Category	Tool	Method
GNN	PyTorch	AGE
		LGNN
		NOCD
Sparse AE	PyTorch	GraphEncoder
Denoising AE	Keras	DNGR
	Matlab	MGAE
Graph Convolutional AE	PyTorch	SDCN
	TensorFlow	O2MAC
Graph Attention AE	TensorFlow	DMGC
		SGCMC
Variational AE	TensorFlow	ARGA/ARVGA
		DGLFRM

Source: Su, Xing, et al. "A comprehensive survey on community detection with deep learning." *IEEE transactions on neural networks and learning systems* 35.4 (2022): 4682-4702.

- The number of communities after grouping can be different from the ground truth
- No clear community correspondence between clustering result and the ground truth
- Normalized Mutual Information can be used



- Entropy: the information contained in a distribution:

$$H(X) = \sum_{x \in X} p(x) \log p(x)$$

- Mutual Information: the shared information between two distributions:

$$I(X; Y) = \sum_{y \in Y} \sum_{x \in X} p(x, y) \log \left(\frac{p(x, y)}{p_1(x)p_2(y)} \right)$$

- Normalized Mutual Information (between 0 and 1):

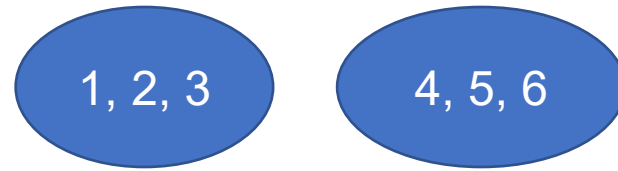
$$NMI(X; Y) = \frac{I(X; Y)}{\sqrt{H(X)H(Y)}}$$

- Consider a partition as a distribution (probability of one node falling into one community), we can compute the matching between two clusters

- Consider all the possible pairs of nodes and check whether they reside in the same community
- An error occurs if:
 - Two nodes belonging to the same community are assigned to different communities after clustering
 - Two nodes belonging to different communities are assigned to the same community
- Construct a contingency table

Clustering Result		Ground Truth	
		$C(v_i) = C(v_j)$	$C(v_i) \neq C(v_j)$
	$C(v_i) = C(v_j)$	a	b
	$C(v_i) \neq C(v_j)$	c	d

$$accuracy = \frac{a + d}{a + b + c + d} = \frac{a + d}{n(n-1)/2}$$



Ground Truth



Clustering Result

		Ground Truth	
		$C(v_i) = C(v_j)$	$C(v_i) \neq C(v_j)$
Clustering Result	$C(v_i) = C(v_j)$	4	0
	$C(v_i) \neq C(v_j)$	2	9

$$\text{Accuracy} = (4+9) / (4+2+9+0) = 13/15$$

Metrics	Overlap	Ground Truth
ACC	✓	Yes
NMI	✗	Yes
Overlapping-NMI	✓	
Precision	✓	Yes
Recall	✓	Yes
F1-Score	✓	Yes
ARI	✓	Yes
Q	✗	No
Extended-Q	✓	
Jaccard	✓	Yes
CON	✓	No
TPR	✓	No

- "Overlap" indicates whether the metric can evaluate the overlapping community.
- "Ground Truth" indicates whether the actual community structure of the network is required.

Source: Su, Xing, et al. "A comprehensive survey on community detection with deep learning." *IEEE transactions on neural networks and learning systems* 35.4 (2022): 4682-4702.

Categories	Datasets	#Nodes	#Edges
Social Networks	Friendship7	68	220
	Football	115	613
	Facebook	1,045	26,749
	LiveJournal	44,093	871,409
	Twitter	87,760	1,293,985
	Orkut	297,691	7,747,026
	DBLP	317,080	1,049,866
	Youtube	1,134,890	2,987,624
Citation Networks	Small-hep ²	397	812
	Polblogs	1,490	16,718
	Cora	2,708	5,429
	Citeseer	3,312	4,732
	Large-hep ²	11,752	134,956
	Pubmed	19,729	44,338
	arXiv	576,000	6,640,000
	US patents	3,700,000	16,500,000
Collaboration Networks	Computer Science	22,000	96,800
	Chemistry	35,400	157,400
	Medicine	63,300	810,300
	Engineering	14,900	49,300
Others	Amazon	334,863	925,872
	Google	875,000	4,320,000

Source: Su, Xing, et al. "A comprehensive survey on community detection with deep learning." *IEEE transactions on neural networks and learning systems* 35.4 (2022): 4682-4702.



네트워크 과학연구실
NETWORK SCIENCE LAB



가톨릭대학교
THE CATHOLIC UNIVERSITY OF KOREA

